Memory Consistency and Visibility Models

Unit 3 – Synchronization and Concurrency Control Design

Prashant Jadav

DEPSTAR – CHARUSAT

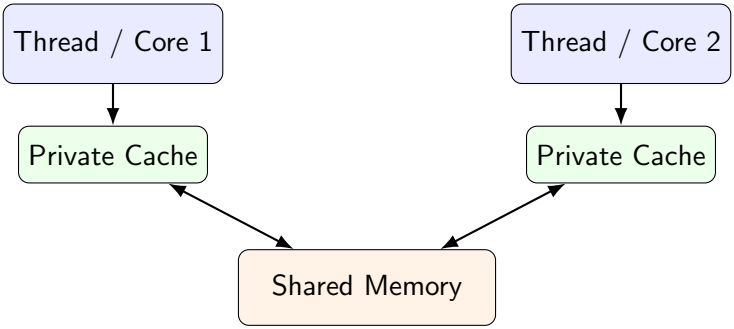
Why Memory Consistency Matters

- Concurrent threads may execute on different CPU cores.
- Each core may have private caches, buffers, and out-of-order execution.
- The order in which a thread performs memory operations is not always the same order observed by another thread.
- Therefore, synchronization requires guarantees about:
  - **Visibility**: when does one thread observe another thread's update?
  - **Ordering**: in what order can memory operations become observable?
  - **Atomicity**: can an operation be observed partially?

Key idea

A memory model defines the contract governing the observable behavior of shared-memory operations.

A Shared-Memory System



Shared variables: X, Y, flags, counters, etc.

- Both threads can access the same logical address space.
- Hardware and compiler optimizations may change when operations become observable.
- Synchronization primitives provide the mechanisms needed to establish ordering and visibility.

Visibility: What Does Another Thread See?

Thread 1

$$X \leftarrow 1$$

- Thread 1 updates shared variable X.
- The update may initially reside in a cache or store buffer.

Thread 2

$$r \leftarrow X$$

- Thread 2 reads X.
- What value is guaranteed to be observed?

Visibility problem

Without an appropriate synchronization/ordering mechanism, observing a write from another thread at a particular point is not guaranteed merely because the write occurred earlier in source code.

Ordering and Reordering

Consider:

$$\begin{aligned} &\text{Thread 1:} \quad X = 1; \quad Y = 1; \\ &\text{Thread 2:} \quad r_1 = Y; \quad r_2 = X; \end{aligned}$$

- Program order in Thread 1 is $X = 1$ before $Y = 1$.
- A compiler or processor may reorder independent operations when permitted by the memory model.
- Another thread may therefore observe a different order unless synchronization constrains the operations.

Design principle

Synchronization is not only about mutual exclusion; it also establishes ordering and visibility guarantees.

Memory Consistency vs. Cache Coherence

Memory Consistency	**Cache Coherence**
Defines allowed ordering/visibility of memory operations.	Maintains a coherent view of a particular memory location across caches.
Concerned with behavior across operations and threads.	Primarily concerned with agreement on the value/order for an individual location.
Affects what executions a programmer may observe.	Helps make multiple cached copies behave consistently.

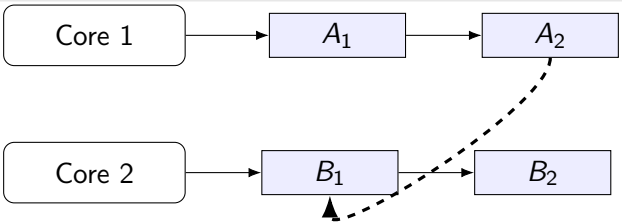
Important

Cache coherence and memory consistency are related, but they are **not the same concept**.

Sequential Consistency

Definition

A system is sequentially consistent if the result of execution is the same as if all operations of all processors were interleaved in a single global sequence, while preserving each processor's program order.



Any global interleaving is allowed, but $A_1 \prec A_2$ and $B_1 \prec B_2$ must be preserved.

Sequential Consistency: Example

Let initially $X = Y = 0$.

$$P_1: \quad X = 1; \quad r_1 = Y;$$
$$P_2: \quad Y = 1; \quad r_2 = X;$$

Under sequential consistency

- The four operations can be interleaved.
- Each processor's local order must be preserved.
- Some outcomes are allowed; some are not.

Why it is useful

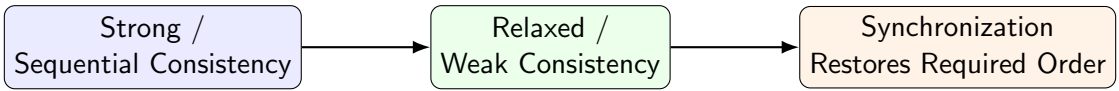
- Simple mental model.
- Easier reasoning about concurrent programs.
- Stronger ordering can restrict optimization opportunities.

Trade-off

Stronger consistency $\rightarrow$ easier reasoning, but potentially greater implementation cost or fewer optimization opportunities.

Relaxed / Weak Consistency Models

- Modern systems may permit more reordering than sequential consistency.
- Relaxed models allow certain operation orderings to be weakened.
- Synchronization operations are then used to restore the ordering needed for correct communication.



Key point

A relaxed model does not mean "anything can happen." It defines a specific set of permitted behaviors.

Common Ordering Relationships

For two operations X and Y, a memory model may constrain relationships such as:

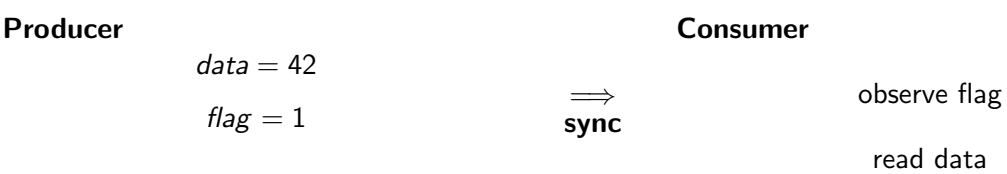
$$R \to R, \qquad R \to W, \qquad W \to R, \qquad W \to W$$

Relation	Meaning
$R \to R$	Read must precede a later Read
$R \to W$	Read must precede a later Write
$W \to R$	Write must precede a later Read
$W \to W$	Write must precede a later Write

Interpretation

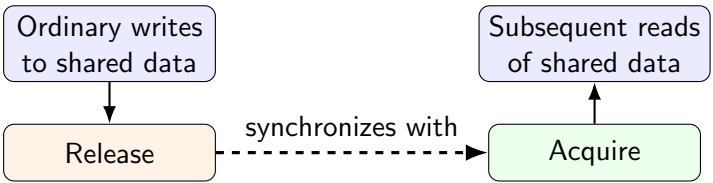
Different consistency models preserve different subsets of these ordering constraints.

Synchronization Creates Visibility



- The synchronization operation establishes a relationship between the producer and consumer.
- The required ordering is typically expressed using **release** and **acquire** semantics.
- The goal is that observing the synchronization event also makes the relevant prior writes observable.

Acquire and Release Semantics



- **Release**: prevents relevant earlier operations from being moved after the release.
- **Acquire**: prevents relevant later operations from being moved before the acquire.
- Together they are commonly used to communicate data safely between threads.

Visibility Through a Lock

Thread 1

1. Acquire lock
2. Write $X = 10$
3. Release lock

Thread 2

1. Acquire same lock
2. Read X
3. Release lock

Synchronization relationship

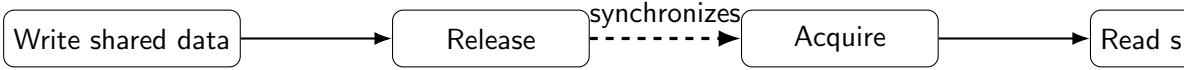
The release of the lock and a later successful acquire of the same lock establish the ordering needed for the protected data to become visible to the acquiring thread.

$$\text{Write } X \longrightarrow \text{Unlock} \longrightarrow \text{Lock} \longrightarrow \text{Read } X$$

Happens-Before: A Useful Reasoning Tool

Concept

A happens-before relationship expresses that one operation is ordered before another operation according to the language/runtime synchronization rules.



- Happens-before is stronger than simply saying "this instruction was written first."
- It is established by defined synchronization mechanisms.
- It is useful for reasoning about visibility and data-race-free programs.

Data Race and Visibility

Data race

Two concurrent threads access the same memory location, at least one access is a write, and the accesses are not properly synchronized according to the applicable memory model.

Unsafe

$$P_1 : \ X \leftarrow 1$$

$$P_2 : \ r \leftarrow X$$

- No defined synchronization relationship.
- Behavior depends on the language and memory model.

Safer design

$$P_1 : \ \text{write } X; \ \text{release}$$

$$P_2 : \ \text{acquire}; \ \text{read } X$$

- Explicit ordering/visibility relationship.

Consistency Models: Design Trade-off

Model family	Ordering guarantee	Reasoning	Optimization freedom
Sequential consistency	Strong global ordering with program order preserved	Easier	Lower
Relaxed/weak models	Selected orderings may be relaxed	More complex	Higher
Acquire/release style	Strong ordering around synchronization boundaries	Practical for synchronization	Higher outside synchronization

System-design perspective

The central trade-off is between **programmability/correctness reasoning** and **performance/optimization freedom**.

Practical Programming Rules

1. Do not assume that source-code order automatically determines cross-thread observation order.
2. Protect shared mutable state with an appropriate synchronization mechanism.
3. Use atomic operations when the shared operation requires atomicity.
4. Use acquire/release or stronger synchronization semantics when communication between threads requires ordering and visibility.
5. Avoid data races rather than relying on accidental hardware behavior.
6. Treat the language memory model as the programming contract; do not assume that one CPU architecture's behavior applies everywhere.

Remember

Correct synchronization must establish the **visibility + ordering + atomicity** guarantees required by the algorithm.

Scenario-Based Analysis

Scenario: A producer writes a result into shared variable *data* and then sets *ready* $= 1$. A consumer waits until it observes *ready* $= 1$ and then reads *data*.

Questions

1. Why is observing *ready* $= 1$ not, by itself, sufficient to reason about the visibility of *data*?
2. Which ordering relationship should connect the producer's data writes with the consumer's data reads?
3. How can a release operation in the producer and an acquire operation in the consumer establish the required relationship?
4. What could go wrong if the communication is implemented without the required synchronization?

Design focus

Analyze the problem in terms of **program order, visibility, synchronization, and happens-before**.

Exam-Oriented Questions

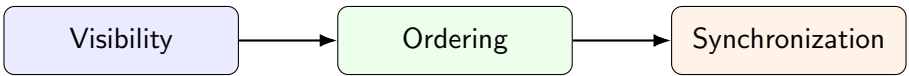
1. Differentiate **memory consistency** and **cache coherence**.
2. Explain visibility and ordering in a shared-memory multiprocessor.
3. Define sequential consistency with a suitable example.
4. Compare sequential consistency and relaxed consistency models.
5. Explain how acquire and release operations help establish visibility.
6. Analyze a producer-consumer communication sequence and identify the required happens-before relationship.
7. Why can compiler and processor reordering create concurrency bugs?

Summary

- **Memory consistency** specifies the allowed observable ordering/behavior of shared-memory operations.
- **Visibility** concerns when one thread can reliably observe another thread's updates.
- **Cache coherence** and consistency solve different aspects of shared-memory behavior.
- **Sequential consistency** provides an intuitive global interleaving model while preserving each thread's program order.
- **Relaxed models** permit selected reorderings to improve performance.
- **Synchronization** establishes the ordering and visibility relationships needed for correct communication.
- **Acquire/release** semantics are important building blocks for synchronization.
- **Happens-before** provides a useful way to reason about visibility and ordering.

Key Takeaway

A concurrent program is correct only when its synchronization establishes the visibility and ordering guarantees it requires.



References

- William Stallings, *Operating Systems: Internals and Design Principles*, 9th Edition.
- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, *Operating System Concepts*, 10th Edition.
- Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, 5th Edition.
- Vijay Nagarajan, Daniel J. Sorin, Mark D. Hill, David J. Wood, *A Primer on Memory Consistency and Cache Coherence*, 2nd Edition.